COMP2200/COMP6200 Practical Exercise (Week 5): k-NN vs Logistic Regression — Hands-on, Orange & Python

Learning goals (today)

- Do a clean train/validation/test split *by hand* and avoid data leakage.
- Compare a linear decision boundary (logistic regression) with non-parametric k-NN ($k=1$ and $k=3$).
- Compute accuracy, precision, recall and F1 from a confusion matrix.
- Read AUC and MCC from Orange's evaluation output.
- See bias/variance in action: 1-NN (low bias, high variance) vs 3-NN (more bias, less variance).
- Reproduce the workflow in Orange and then inspect the same train/test-split idea in Python.

Kit you need

Coloured tokens (red/green/blue/purple), rulers or thread, a dinosaur or two, Blu-Tak (optional), and a phone/camera for photos.

Example Week 5 prac kit.

1 Part A — Classification with coloured tokens (by hand)

Setup (5 minutes)

A1. Pick 2–3 colours as classes (e.g. red, green, blue). One person scatters reds, another greens, another blues, so each colour tends to cluster, but still has a bit of overlap. Take a quick photo.

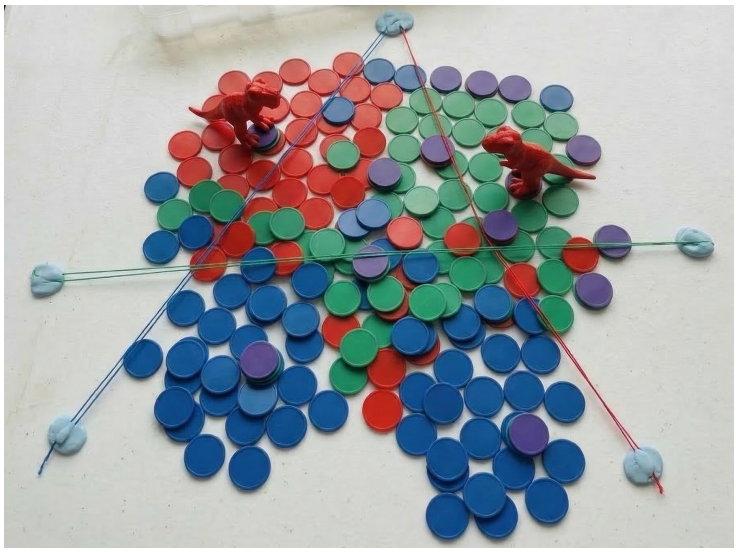
A2. **Split:** However many dinosaurs you have, that is how many test instances you have. Put a dinosaur at random on some tokens and try to make it cover up the colour of the token. Make another 5 of your tokens be validation tokens. Come up with a scheme to identify them uniquely, e.g. "the validation token at the top right". Put a purple token on them so that you can't see what colour it is underneath. Everything else is training data.

One possible end state for A2: dinosaurs mark the test tokens, purple tokens mark the validation tokens, and everything else is training data.

A3. *Fun extension:* Split the blue tokens into two well-separated piles. This will stress-test linear boundaries.

Logistic regression by ruler or thread (10 minutes)

A4. Using only **training** tokens, place a ruler or a stretched piece of thread as a linear decision boundary for **red vs not-red**. If you have spare rulers, also set up **green vs not-green** (one-vs-rest boundaries).



One possible setup after A4: thread marks linear decision boundaries on the training layout. This is a pretty good boundary for blue, but maybe a bit cautious for red and green.

A5. Validate: For each **validation** (purple) token, predict the class by which side of the relevant ruler/thread boundary it falls on (for 3 classes, compare boundaries or pick the nearest side). Record predictions without lifting the purple tokens.

A6. When finished, **reveal** the validation labels (peek under purple) and build a confusion matrix. Compute per-class precision/recall/F1 and overall accuracy (definitions below). Photograph the setup and your matrix.

k-NN ($k=1$ and $k=3$) on the same validation set (15 minutes)

A7. Remove the ruler/thread boundaries. Using only **training** tokens, run 1-NN: for each validation token, find the **single** nearest training token (use a ruler or a piece of thread if disputed) and predict its colour. Record predictions; then reveal and score.

A8. Repeat for 3-NN: use the **three** nearest training tokens and take the majority colour (break exact ties by coin flip or nearest of the tied colours).

A9. Model selection: Compare validation metrics for Logistic (ruler), 1-NN, and 3-NN. Pick the winner (be honest: if two are basically tied, state that).

Now (and only now) test once (5 minutes)

A10. Using the **chosen** model, predict the **test** tokens. Lift the dinosaur, reveal labels, and compute the test confusion matrix and metrics.

A11. Do not adjust anything after seeing the test results. If you do, you've leaked. Full stop.

Metric definitions (for each class c; one-vs-rest)

TP = true positive
TN = true negative
FP = false positive
FN = false negative

$$\text{Accuracy} = \frac{TP + TN}{N},$$

$$\text{Precision}_c = \frac{TP_c}{TP_c + FP_c},$$

$$\text{Recall}_c = \frac{TP_c}{TP_c + FN_c},$$

$$\mathrm{F1}_c = \frac{2 \cdot \mathrm{Prec}_c \cdot \mathrm{Rec}_c}{\mathrm{Prec}_c + \mathrm{Rec}_c}.$$

Report macro-F1 (average F1 over classes) and/or per-class numbers.

Common gotchas

- If blue is split into two piles, a single straight boundary is **wrong-shaped**. Expect k-NN to outperform the ruler here.
- If classes are very imbalanced, raw accuracy can mislead; look at per-class recall.
- Tie-breaking for 3-NN: decide the rule *before* scoring.

2 Part B — Reproduce in Orange (required; ∼25 minutes total)

Part A used an explicit train/validation/test split because you were doing model selection by hand. In Orange, `Test & Score` can automate that comparison by running repeated validation folds for you, which is why this section switches to cross-validation.

B1. **Paint the scene:** Open Orange → drag `Paint Data`. In the canvas, recreate your token layout from Part A. Use distinct colours for classes (discrete target).

B2. Add `Select Columns`. Ensure your painted class is the **Target**.

B3. Add two learners: `kNN` (set $k \in \{1, 3\}$; you can duplicate the widget) and `Logistic Regression`.

B4. Add `Test & Score`. Connect `Paint Data` to it as Data, and the learners as Learners.

B5. In `Test & Score`, choose **Cross validation (5-fold)**. Run once with $k = 1$ and once with $k = 3$ (or put both kNN widgets in at once). Record Accuracy, F1, AUC, and MCC.

B6. Open `Confusion Matrix` from `Test & Score` to see per-class behaviour. Also open `Scatter Plot` and colour by the predicted class to eyeball the boundary shapes.

B7. *If time:* Open `ROC Analysis` from `Test & Score`. This lets you see the ROC curve that the AUC score comes from.

B8. *Optional hyperparameter tuning (if time):* Insert `Parameter Fitter` for kNN with $k \in \{1, 3, 5, 7\}$. Feed its output into `Test & Score`. This is the principled version of the A-part validation you did by hand.

3 Part C — Python notebook (10–20 minutes, or finish later this week)

Use the provided notebook `Week_5_Train_Test_Split.ipynb`. Open it either in Jupyter Notebook on your own machine or in Google Colab. This part keeps the Python workload deliberately small. It mirrors the lecture: load a CSV with pandas, split it into train and test tables, and see what `random_state` and `test_size` do.

C1. Open the notebook and fill in the missing `TODO` cells as you go.

C2. Load the Telco churn CSV into a pandas DataFrame and inspect a few columns.

C3. Run `train_test_split(df)` twice. What changes from one run to the next?

C4. Add `random_state=42`. What stays the same now?

C5. Add `test_size=0.2`. How many rows end up in the test table?

C6. Answer the reflection questions at the end of the notebook, including: which column would later be the target if we wanted to build a churn classifier?

Debrief prompts (discuss briefly)

- When blue was split into two piles, how did logistic fare vs k-NN? Why?
- Did 1-NN overreact to local clutter? Did 3-NN smooth it out?
- In the notebook, why did the split change from run to run until you set `random_state`?
- If you wanted to build a churn classifier later, which column would be the target?

Practice quiz questions

Proceed to iLearn and do some practice quiz questions.